Query Processing and Optimization Techniques using MySQL and MongoDB

Liang Yrenn Da
Faculty of Engineering and Technology
No. 5, Jalan Universiti, Bandar
Sunway, 47500, Selangor Darul Ehsan,
Malaysia
22067128@imail.sunway.edu.my

Yeoh Jie Long
Faculty of Engineering and Technology
No. 5, Jalan Universiti, Bandar
Sunway, 47500, Selangor Darul Ehsan,
Malaysia
22071112@imail.sunway.edu.my

Ooi Jin Yon
Faculty of Engineering and Technology
No. 5, Jalan Universiti, Bandar
Sunway, 47500, Selangor Darul Ehsan,
Malaysia
22039069@imail.sunway.edu.my

Abstract

Database management systems (DBMS) are made up of query processing and optimization, which directly affect the performance, scalability, and effective use of resources of the system. With the increase in the size and complexity of datasets as a result of modern data-driven applications, efficient query implementation systems are also needed to guarantee accessibility to data promptly and without failure. The paper provides a systematic literature review on query processing and optimization, both in the traditional DBMS setting and in current distributed and large-scale databases.

This study aims at reviewing and synthesizing existing methods of query processing and optimization and exploring the use of the methods in centralized and distributed database systems. This paper initially explains essential query processing functionality in a DBMS, such as SQL query transformation into logical and physical execution plans, typical join processing methods, data access methods such as sequential scans and index scans. These ideas give one an idea of the way queries are internally executed, and how efficiency in executions can be enhanced.

Following up, this paper then further elaborates on this argument to query processing and optimization in distributed and large-scale databases, where data partitioning, parallel query execution, query shipping, cost of network communication, and scalability issues become the matters of greater concern. This paper will identify the performance trade-offs and current limitations by examining modern peer-reviewed literature on how traditional query processing methods can be adapted to utilize large datasets and distributed architectures as well as the modification process of traditional query processing methods under large-data conditions. The findings establish a clear foundation for understanding current approaches to query processing and optimization and their relevance in modern database systems.

Keywords: Query Processing, Query Optimization, DBMS, Distributed Databases, Parallel Query Execution, Large-Scale Databases

I. INTRODUCTION

A. General Background

The current digital economy has led to the generation of huge volumes of transactional data daily in E-Commerce sites. Database Management Systems (DBMS) serve as a key foundation of storing this data, it is not only important to store this data but also to retrieve it in an efficient manner. With historical transaction logs in the millions of rows, standard SQL queries used in reporting such as computing monthly revenue or identify trend on popular products, may experience significant latency. Query Optimization refers to the organized procedure of adjusting queries and database structures to reduce the time needed and resource consumption of executing queries to ensure that real time business analytics can be cost effective [1].

B. Problem Statement

The specific challenge tackled in this report is the decline in performance of analytical queries on the increasing retail databases. Using the UCI Online Retail Dataset to document transactions of an online wholesaler in the UK, we observe that with the size of the dataset that is simulating many year's complex queries and queries restricted by date or customer have become inefficient. Without optimization, these queries rely on "Full Table Scans", which focus on the database engine to read every transaction row in order to respond to simple business queries and this leads to unacceptable wait time to end-users.

C. Significance of the Problem

The issue of query latency is not only a technical management problem, but also a business requirement. Delays in the data retrieval process may affect the decision-making process and add to the negative user experience in the e-commerce industry. For example, a dashboard that requires 10 seconds to open "Sales by Country" prevents managers to be responsive to the trends in real time. Moreover, ineffective queries use up too much CPU and I/O memory, this could lead to a raising of cost to operation cloud-hosted database infrastructure [2]. Therefore, it is necessary to implement optimization methods such as indexing and query rewriting to ensure that the systems are scalable, cost effective and responsive.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

D. Objective

The primary objective of this report is to analyse and improve the performance of read-heavy analytical queries within the context of the Online Retail dataset. Specifically, this study aims to:

1. Establish a performance baseline for common retail queries (e.g., RFM Analysis, Revenue Reports) on a large-scale dataset.
2. Implement specific optimization strategies, including B-Tree Indexing and Query Restructuring.
3. Quantify the improvement by comparing execution time (latency) and resource cost before and after optimization.

E. Scope of the report

This analysis utilizes the **MySQL 8.0** and **MongoDB** Relational Database Management System. The data used in the study is the Online Retail dataset that has been scaled up through data inflation methods to emulate a large-volume production model of more than 4 million transaction rows. The scope is aim to optimization Read (SELECT) operations that are associated with data warehousing and reporting as it does not include write-optimization and distributed database architecture.

II. Literature Review

This literature review focuses on query processing and optimization techniques in database management systems on how these techniques improve query execution efficiency, system performance, and resource utilization. As modern databases continue to handle large volumes of structured and semi-structured data, efficient query processing has become increasingly critical to ensure timely data retrieval and support decision-making processes.

A. Query Processing Techniques

The query processing is one of the main tasks on a database management system (DBMS) that converts high level declarative SQL query into low level code that can be executed directly. Performance is very important in the system, especially in the large-scale and distributed systems where latency, throughput and memory usage are major limitations. The following literature review synthesizes new progress in query including pipeline scheduling, execution models, and join processing, scan techniques, and distributed query frameworks.

The emphasis was laid on the research with significant theoretical contribution, experimental confirmation, or system-level innovations in query processing. Each of the selected studies was directly applicable to one or more of the following themes: query pipeline execution, execution models, join algorithms, data scanning strategies, and distributed query execution.

1) Query Execution Pipeline

The research by Landgraf et al. [3] is about optimizing the memory used when executing a query using the pipeline. The authors present the following problem: how to schedule the operators of the pipelining to reduce the maximum memory consumed by the intermediate results. They offer a mathematical model and offer four strategies of scheduling such as the exact and heuristic solutions. Their experiments using the Join Order Benchmark demonstrate that these techniques can significantly reduce memory usage while incurring minimal planning overhead. The work is particularly applicable in the case of in-memory databases; the performance of such databases is directly influenced by memory efficiency.

2) Execution Models

Zhang et al. [4] suggest the cloud-native hybrid transactional and analytical processing system AnalyticDB-PG deployed in Alibaba Cloud. The system combines both columnar and row-based data structures and also includes Laser which is an execution engine that supports both the execution of vectors as well as just-in-time (JIT) compilation. Laser enables fine-grained operator fusion and employs runtime profiling for execution plan optimization, resulting in superior performance across mixed workloads. Wagner et al. [10] propose a new architecture named Incremental Fusion that integrates compiled and vectorized query processing. The system produces both a native code and an interpreter in sub operators to enable switching during runtime with characteristics of the workload. Schiavio et al. [11] improve SQL query compilation in Apache Spark by dynamically optimizing compiled code. Their speculative approach profiles data access patterns at runtime and selectively recompiles code paths, leading to significant performance gains on diverse formats like CSV and JSON.

3) Join Processing Techniques

Dynamic hybrid hash join design should take into account the partitioning, spilling, and growth strategies in the presence of uncertain data statistics [5]. Those refinements proposed, tested in AsterixDB, show lower I/O and increased skew stability. The article "Sort-merge join on multi-GPU systems: Heterogeneous sorting and CPU-GPU coordination" describes the advantages of multi-gpu systems [6]. Their technique has a high scale and performance with the use of the sorting and merging with the help of the GPU interconnects and CPU.

4) Scan Techniques

Skip index enables efficient authenticated inter-block queries in blockchain systems by supporting logarithmic-time lookups with cryptographic guarantees [7]. Although proposed outside traditional DBMSs, the work is conceptually relevant as it illustrates general scan-reduction principles through hierarchical indexing. Pando uses predicate correlation to provide better data skipping through multiple logical layouts to achieve better scan-efficiency [8]. The framework goes beyond the traditional min-max indexing by matching a predicate selectivity with the data storage.

5) Distributed and Adaptive Query Execution

Ratchet supports resource reallocation and pre-emption, enabling state-loss-free adaptive query

suspension and resumption in elastic cloud environments [9].This structure is needed to cloud-native and serverless DBMSs, which respond to cost-efficiency and resilience to dynamic workloads.

Synthesis and Gaps More recent works focus on mixed implementation approaches between vectorization and compilation [4][10][11]. This type of integration enhances performance based on a wide range of query loads. The developments in the field of join processing demonstrate significant progress in terms of CPU acceleration and powerful hash joins [5][6] but challenges such as skew management remain. Intelligent scanning strategies [7][8] reduce the amount of data accessed unnecessarily but depend largely on optimum layout of data and indexing overheads. Resource-aware solutions such as pipeline scheduling [3], adaptive cloud execution [9] are not comprehensive, but some systems to optimize latency, energy, and cost in serverless settings are uncommon. It can be advanced in the future by studying real-time, multi-objective optimization and dynamical operator optimized.

In conclusion, this review discussed modern developments in query processing with respect to pipeline optimization, execution model, join evaluation, scan strategy, and distributed execution. Critical work has been in memory-conscious scheduling [3], hybridized engines based on both vectors and compiled code [4][10], adaptive speculative compilation [11], scalable to the GPU join architecture [6], and intelligent skipping implementations [7][8]. Although these methods increase performance and efficiency, there are still issues of incorporating adaptive optimization into elastic and cloud-native environments [9]. These gaps can be the next step in query processing research.

B. Query Optimization Techniques

Query optimization is the DBMS process that aim to reduce query execution through effective plan. Early research emphasized the importance of join order optimization as it brings significant impact on intermediate the result size and overall performance [1]. The research demonstrated that poor join ordering can lead to excessive resource consumption. By using dynamic programming and pruning strategies, their study demonstrated that systematic exploration of join orders enables more efficient execution plans, especially for complex multi-join queries.

Besides that, cost-based optimization has further investigated as a primary mechanism for selecting efficient query execution plans. A group of researchers had studied cost-based query optimization in multistore database systems, where queries span heterogeneous data engines with varying performance characteristics [2]. They proposed to generate multiple candidate execution plans and select the optimal plan based on cost estimation. As the result, it showed that accurate cost modelling and global plan evaluation significantly improve query execution efficiency in heterogeneous environments.

In addition, one of the research papers also explored rule-based optimization and query rewriting to simplify queries before people use. They have examined how logical transformations can rewrite queries into equivalent but more efficient forms [3]. They had addressed challenges related to rewriting explosion and proposed pruning techniques to reduce redundant rewritings. As the result, they find effective query rewriting able to reduces optimization overhead and complements cost-based techniques by improving the structure of the logical query.

Moreover, query rewriting strategies have been compared against alternative reasoning approaches in data integration and ontology-based data access scenarios. One research had introduced a benchmarking framework for evaluating forward-chaining and backward-chaining techniques [4]. Their experimental results showed that query rewriting remains competitive for efficient query answering when preprocessing and indexing costs are amortized across multiple queries. This work provides empirical evidence that optimized query rewriting techniques play a crucial role in query optimization for complex integration environments

However, the other research finds the limitations of cost-based query optimization caused by inaccurate cardinality estimation and plan enumeration strategies. They provided a comprehensive survey on the core components of DBMS query optimizers, namely cardinality estimation, cost models, and plan enumeration [5]. Their study showed that estimation errors can happened starting from the optimization pipeline. This led to suboptimal execution plans especially for complex multi-join queries. This highlight that improvements in these foundational techniques are important for enhancing optimizer reliability and efficiency. Thus, they highlight the importance of accurate statistics and systematic plan exploration as key factors in effective query optimization.

In conclusion, the research find that effective query optimization relies on a join order optimization, cost-based plan selection, and query rewriting. They highlight the critical impact of join ordering on intermediate result size. At the same time, the other study shows that cost-based optimization helps to enables systematic evaluation of alternative execution plans. Moreover, complementary rule-based and query rewriting techniques enhance optimization by simplifying the logical query structures. However, the effectiveness of this technique is strongly influenced by the accuracy of cardinality estimation and plan enumeration. In short, these studies indicate that robust cost models, accurate statistics, and structured plan exploration remain fundamental to achieve efficient query optimization in DBMS.

C. Query Processing and Optimization in Distributed and Large-Scale

Modern large-scale database systems increasingly adopt cloud-native architectures that separate computation and storage and exploit elasticity. An example is PolarDB-X, a distributed relational engine based on PolarDB, which is developed by Alibaba, and that PolarDB-X is said to have a lot of the cloud-native properties, including the

multi-datacentre deployment and elasticity. PolarDB-X allows scaling widely: the authors report that it can easily scale the cluster to meet a transient surge in traffic. Similarly, PolarDB Serverless is specifically configured to work with disaggregated cloud data centres: its model decouples the CPU resource on compute nodes with the remote memory pool and storage pool, so that each resource pool could be scaled up or down as needed, and it offers on-demand provisioning at various dimensions. These dynamic capabilities of provisioning provide quicker recovery benefits as well as elastic throughput under load. In general, this trend can be seen as the process of moving towards monolithic designs to systems in which it is possible to add nodes or redeploy tenants on-the-fly. According to Bruno et al. [12], this change in data warehousing at Microsoft has been as follows: the shift towards the cloud, separation of compute/storage, adoption of serverless components has necessitated a modification of the query optimizer within Fabric DW. Likewise, exploiting native cloud capabilities, systems such as veDB-HTAP extend OLTP with OLAP using the Secondary Engine of MySQL, and have a highly integrated system architecture and a query interface. To conclude, the modern query engines are designed based on cloud-native (multi-datacentre replication, shared-storage or disaggregated storage, HTAP support) components that allow workloads to be transparently scaled and integrated.

Many of the new techniques have their basis in elasticity. Liu et al. [9] call the necessity to adapt to changing workload and resource availability quickly but note that the volatile cloud resources (e.g., spot VMs or serverless functions) demand new primitives, e.g. query pre-emption and resource arbitration. They present Ratchet, a resource-adaptive query execution system which enables query suspension and resumption at different granularities, and fine-grained pricing to take advantage of low-cost transient VMs without incurring overage costs. Event driven architectures can provide highly transparent elasticity. An example is an architecture-less DBMS in which data and control are streams and elasticity is achieved by directing events to a variable number of compute units (ACs) [16]. They observe that with flexible event routing and data beaming there is an opportunity to have transparent elasticity without introducing any extra latencies, and streams can be rerouted or repartitioned to spread load in the system on an adaptive basis. It implies that, as demand increases, the system merely adds more ACs and reroutes events without being repartitioned manually. Practically, serverless query engines such as Starling [17] take advantage of such elasticity: Perron et al. note that Starling maps tasks to function invocations such that users only pay the amount of compute resource its query actually requires, and that the number of invocations can increase and decrease accordingly each time a query is run. Stated differently, Starling uses as many Lambda functions as required (sub-second startup time) to scan and combine data in cloud storage. This on-demand provisioning is virtually unlimited parallel at very fine granularity with low latency even on large datasets. In fact, Perron et al. observe that cloud platforms enable the use of arbitrary numbers of small functions, with extremely low startup times, and with approximately no bounds to parallelism, such that it is practical to perform parallel database operators in a serverless manner.

Besides, it is also essential to execute queries on a distributed system among many nodes. Many modern systems use massively parallel processing (MPP) and distributed optimization. An example, PolarDB-X, an MPP model of analytical (OLAP) queries can be executed with many coordinator (CN) nodes sending work to many database (DN) nodes (both row and column stores) and allowing parallel query pipelines. It also improves performance as it drives the operators to the storage layer where they are deemed to fit. In particular, the optimizer of PolarDB-X will push parts of query (Join, Agg, Sort operator) to where the corresponding storage nodes to do near-data computing to minimize data movement. The Microsoft Fabric Data Warehouse also developed its optimizer to execute MPP on the cloud: the frontend generation creates a global space of the distributed plans, the distributed generation evaluates alternative data distribution schemes to reduce the movement, and each backend node also optimizes its own local fragment of the plan. These hierarchical optimization phases make the final execution plan an efficient end-to-end plan. New architectures such as AnyDB [16] even cross the boundary between nodes: in AnyDB, operations and data are carried in event streams such that any computing node can process any element of a query. In practice, in an experiment, prefetching data (also known as beaming) enables the execution of a query to be overlapped with compilation, so that the overall runtime is almost comparable to the time spent on optimization. AnyDB demonstrates that in so doing, queries can be executed on disaggregated compute with apparent performance cost. Overall, DQP has become globalized in terms of optimisation and elasticity in node allocation, allowing both scale-out and dynamic workload balance.

1) Adaptive query optimization

Systems increasingly exploit runtime information to refine plans. For instance, veDB-HTAP [13] implements a two-fold adaptive strategy: the query execution dynamically collects and uses statistics to improve runtime performance, and it monitors resource usage to maintain quality-of-service under load. According to Chen et al., the adaptive query execution of veDB-HTAP could be of two types: (1) adaptive query execution that gathers and uses dynamic runtime statistics to improve query performance; (2) adaptive query execution that uses the information of available runtime resources to provide a high quality of service despite heavy load. This is enhanced by its intelligent query routing: instead of fixed rules, veDB-HTAP relies on a cost-based and machine-learned router to choose the most appropriate engine (OLTP or OLAP) to execute a query. On the same note, Ratchet [9] dynamically suspends or reinstates operators on the fly whenever pre-emptions are experienced to ensure loss of transient resources does not bog down the entire query. In other systems, which are not cloud-native, adaptivity has been useful: e.g., refinements of optimizers (or re-optimization of sub plans) in response to partial answers (in work outside

our ten references). One recent systems paper is aimed at execution efficiency: Wagner et al. [10] note that modern engines use either a vectorized batch pipeline or just-in-time (JIT) codegen, both of which perform well under different conditions. They suggest hybridizing the two paradigms and introducing Incremental Fusion, a hybrid engine that dynamically switches between the two paradigms of vectorized interpretation and operator-fusing code generation. Their system achieves competitive performance in both short query performance and long-running workload by generating a system-generated vectorized interpreter based on the compiled code and thus selecting (at runtime) between an interpreted mode of execution and a fused loop mode of execution. These adaptive methods, whether adaptive plans or adaptive execution engines, are used to make sure that systems are efficient to work with various and evolving workloads.

2) Accelerators and parallel algorithms

Parallel algorithms and accelerators are also important in the optimization of large-scale queries. On the one hand, parallel clusters of CPUs and distributed algorithms are conventional (e.g., distributed hash joins, parallel aggregations). Conversely, heterogeneous hardware is being used in modern research. Bang et al. [8] describe how their stateless, event-based model enables incorporation of "heterogeneous compute resources per query, comprising but not restricted to accelerators (e.g., FPGAs or GPUs)". Practically, it implies that a query execution pipeline would be able to send some operators to GPUs but not upset the rest of the engine. An example is given by Maltenberger et al. [6], who introduce a new sort-merge join, which is distributed across various GPUs on a single node (NVLink/NVSwitch). Their multi-GPU join algorithm defeats the normal restrictions of the GPU joins, it allows peer to peer exchange of information, and it is able to cope with out-of-core information. It gives dramatic experimental gains: it has been found to outperform state-of-the-art CPU and GPU baselines, specifically 15.2x and 5.5x faster respectively, with representative TPC-H workloads. Besides this, they demonstrate that the algorithm is linearly scaled with the number of GPUs and that skewness in data do in fact enhance performance. These findings highlight that orders-of-magnitude gains can be achieved using the exploration of the GPU parallelism with computationally intensive query operators. Other works are very fine-tuned to SIMD and concurrency (on the CPU side, it is), but we can only find ten of them. Overall, the literature speaks of a tendency towards employing both distributed parallelism between nodes and fine-grained parallelism (GPUs, FPGAs) within nodes to speed up query processing.

3) Serverless query execution

The rise of FaaS platforms (AWS Lambda, Azure Functions, etc.) has inspired fully serverless databases. Perron et al. [17] developed Starling, based on serverless platforms and cloud functions as its execution fabric. The design by Starling explicitly uses the pay-per-query model: the intermediate results have materialized in cloud storage, and users are charged only based on the computer time on the Lambda. This results in low-cost elasticity: it is demonstrated that Starling is cheaper than classical provisioned clusters at low query rates and still delivers interactive latency on large datasets. Sudhir et al. [8] approach it complementarily, adjusting data layouts to cloud analytics: their system Pando employs correlation-aware partitioning to optimize block skipping in cloud warehouses. Using Pando, the number of irrelevant blocks scanned to execute a query is significantly lower, which results in the best results of up to 2.8x fewer blocks scanned and up to 2.3x faster end-to-end query execution time compared to any previous automated design method. The developments take advantage of software-level cloud storage and elasticity in computing. However, it is important to note that several existing cloud database offerings expose elastic interfaces while still relying on largely fixed underlying architectures; as Bang et al. observe, services such as Aurora Serverless or Snowflake provide elasticity at the service layer rather than through fine-grained execution disaggregation. Real serverless engines such as Starling or applicant designs such as AnyDB target more finer-grained scaling. All in all, the work under survey illustrates that by reconsidering the stack, i.e., disaggregated cloud architectures all the way to query engines and optimizers, modern distributed databases can be highly performant in terms of elasticity, parallelism, and adaptivity, both in theory and in rigorous experimental validation.

Altogether, the mentioned literature demonstrates a meeting point between cloud-oriented design and traditional query optimization. Besides, key contributions include cloud-native architectures with disaggregated resources and rapid scaling, adaptive execution frameworks that pre-empt and redistribute queries in response to volatile resources, massively parallel engines for both CPUs and GPUs, and new paradigms for serverless query engines. All of these works combine to show both theoretical developments (e.g. new optimization principles and execution models) and experimental systems which prove those concepts on large-scale workloads, moving the state of the art in distributed query processing and optimization.

D. AI-Driven Query Optimization

Traditional query optimization is heavily relied on using rule-based heuristics and manual designed cost models to select an efficient optimization plan. However, they often struggle with issues like complex queries, correlated predicates, and dynamic workloads. As the AI keep become advance, recent studies have increasingly explored AI-driven query optimization by implement machine learning and reinforcement learning techniques are integrated into the query optimization pipeline to enhance adaptability and performance.

One of the AI-driven query optimizations is the learning-based optimizer that learns from past query execution experiences. One research proposed a constraint-enhanced learned query optimizer known as Celo [23]. It integrates semantic equivalence among queries into the optimization process. Unlike the other learning-based

optimizers, Celo will identify equivalent SQL queries using denial constraints and augments candidate query plan with high-quality plans derived from equivalent queries. Experimental results on the real-world datasets show the approach did significantly improve query latency and cost compared to the traditional one.

The other study emphasized that hybrid AI-assisted framework that augment rather than classical cost-based optimizers. One research had introduced an AI-powered query optimization framework that integrates learned cardinality estimation, neural cost correction and reinforcement learning-based planning into an existing optimizer architecture [24]. By using feedback by observing the row counts and runtimes, the system continuously adapts to workload changes while maintaining safety through uncertainty gating and fallback mechanisms. The empirical evaluations using benchmark workload had shown a consistent reduction in tail latency and improved plan stability.

AI-driven query optimization also used in multitenant database systems, where the resource contention and workload diversity pose some significant challenges. One research has examined how AI technique enables tenant-aware optimization in shared database infrastructure [25]. By incorporating reinforcement learning, AI-based optimizers can dynamically adapt execution strategies to tenant behaviour and service-level requirements. This study found the improvements in performance isolation, resource utilization, administrative efficiency and demonstrate the suitability of AI optimizer for a cloud-based multitenant environment.

Recent research has further explored reinforcement learning and graph-based approaches for large-scale query optimization. One research proposed a hybrid AI-based framework that combines reinforcement learning with graph neural networks to optimize join order selection and cardinality estimation [26]. By modelling database schemas as graphs and employing proximal policy optimization, the proposed approach achieved substantial reductions in query execution time, CPU and memory usage, and cardinality estimation error when compared to both traditional cost-based and existing machine learning–based optimizers. These findings suggest that combining structural database knowledge with learning-based decision making can significantly enhance optimization effectiveness in large-scale and dynamic environments

In addition, AI have also applied to real-time query processing and task scheduling. One research has optimized task-scheduling for localized big data real-time query processing using machine learning methods such as support vector machines and k-nearest neighbours combined with fuzzy evaluation [27]. Their result found that reduction in query processing time and overall system cost. In short, this shows AI help to improve efficiency and reduce cost.

Overall, the existing studies indicate that AI-driven query optimization provides clear advantages compared to traditional optimization techniques by improving adaptability, accuracy, and performance under complex and heavy workloads. However, there still some challenges remain in areas such as training overhead and seamless integration with legacy database systems. These gaps highlight the need for further research into scalable, interpretable, and safe AI-assisted optimization frameworks.

E. Comparison Table

TABLE I. COMPARISION & SUMMARY OF LITERATURE REVIEW TABLE

Reference	Problem Statement	Methodology	Key Finding
	A. AI-Driven Query Optimization		
Landgraf et al. (2022) [3]	Inefficient scheduling of pipelined query operators leads to high peak memory consumption due to large intermediate results.	Mathematical modelling of pipeline scheduling with exact and heuristic strategies, evaluated using the Join Order Benchmark.	Memory-efficient scheduling significantly reduces peak memory usage with minimal planning overhead, particularly beneficial for in-memory DBMSs.
Zhang et al. (2025) [4]	A single execution paradigm is insufficient to efficiently support mixed OLTP and OLAP workloads in cloud environments.	Design of AnalyticDB-PG with the Laser execution engine combining vectorized execution, JIT compilation, and runtime profiling.	Hybrid execution achieves strong and stable performance across diverse transactional and analytical workloads.
Jahangiri et al. (2022) [5]	Hybrid hash joins are sensitive to skew and inaccurate statistics, causing excessive I/O and performance degradation.	Analysis and refinement of partitioning, spilling, and growth strategies for dynamic hybrid hash joins, evaluated in AsterixDB.	Improved robustness to data skew and reduced I/O costs under uncertain data distributions.
Maltenberger et al. (2025) [6]	CPU and single-GPU join algorithms do not scale efficiently for very large datasets.	Design of a multi-GPU sort-merge join exploiting CPU–GPU coordination and high-bandwidth GPU interconnects.	Achieves significant performance gains and near-linear scalability across multiple GPUs compared to CPU and single-GPU baselines.
Loporchio et al. (2025) [7]	Efficient querying of blockchain data requires reducing scan overhead while preserving cryptographic authentication guarantees.	Introduction of Skip Index, a hierarchical authenticated indexing structure enabling logarithmic-time inter-block queries.	Demonstrates effective scan reduction with authentication; conceptually relevant to scan minimization though outside traditional DBMSs.
Sudhir et al. (2023) [8]	Cloud data warehouses perform unnecessary data scans due to limited exploitation of correlations among query predicates.	Proposal of Pando, using correlation-aware logical partitioning and optimized physical data layouts.	Reduces the number of scanned blocks and improves query execution time beyond traditional min–max indexing techniques.

Liu et al. (2024) [9]	Query execution in cloud-native systems is inefficient under volatile and ephemeral resource availability.	Design of Ratchet, a resource-adaptive execution framework supporting query suspension, resumption, pre-emption, and resource arbitration.	Enables cost-efficient, resilient, and adaptive query execution in cloud-native and serverless environments.
Wagner et al. (2024) [10]	Vectorized and compiled query execution models each perform well only under specific workload conditions.	Development of Incremental Fusion, generating both native code and vectorized interpreters with runtime switching.	Combines low-latency execution and high throughput by dynamically selecting the optimal execution mode at runtime.
Schiavio et al. (2020) [11]	Static SQL compilation in Apache Spark cannot adapt to runtime data access patterns and data formats.	Dynamic speculative SQL compilation using runtime profiling and selective recompilation of code paths.	Significantly improves SQL query performance for heterogeneous data formats such as CSV and JSON.
B. Query Optimization Techniques			
Wu et al. (2003) [18]	Inefficient join ordering leads to large intermediate results and high execution cost	Dynamic programming with pruning for joins order enumeration	Systematic join order exploration significantly improves performance for multi-join queries
Forresi et al. (2023) [19]	Query optimization in multistore systems is challenging due to heterogeneous data engines	Cost-based plan generation and global cost evaluation	Accurate cost modelling improves execution efficiency across heterogeneous environments
Gottlob et al. (2014) [20]	Query rewriting can cause rewriting explosion and high optimization overhead	Rule-based logical transformations with pruning strategies	Effective query rewriting reduces optimization overhead and improves logical query structure
Alhazmi et al. (2022) [21]	Performance comparison between forward-chaining and backward-chaining query answering	Benchmarking framework for query rewriting and reasoning approaches	Query rewriting remains competitive when preprocessing and indexing costs are amortized
Lan et al. (2021) [22]	Inaccurate cardinality estimation leads to suboptimal query plans	Survey of cardinality estimation, cost models, and plan enumeration	Accurate statistics and systematic plan enumeration are critical for reliable query optimization
C. Query Processing and Optimization in Distributed Databases			
Maltenberger et al. (2025) [6]	Traditional CPU and single-GPU join algorithms fail to efficiently process very large relations and do not fully exploit modern multi-GPU interconnects, especially for out-of-core data.	Design and implementation of a heterogeneous multi-GPU sort-merge join using NVLink/NVSwitch with peer-to-peer GPU communication, combined with extensive experimental evaluation on TPC-H workloads.	The proposed multi-GPU join outperforms state-of-the-art CPU and GPU baselines by up to 15.2× and 5.5× respectively, scales linearly with the number of GPUs, and benefits from data skew.
Sudhir et al. (2023) [8]	Cloud data warehouses waste I/O resources by scanning irrelevant data blocks due to limited exploitation of correlations among query predicates.	Proposal of Pando, a correlation-aware logical partitioning and physical data layout optimization framework, evaluated using benchmark and real-world workloads.	Pando reduces the number of scanned blocks by up to 2.8× and improves end-to-end query execution time by up to 2.3× compared to existing automated layout techniques.
Liu et al. (2024) [9]	Cloud-native databases struggle to adapt efficiently to volatile and ephemeral cloud resources such as spot VMs and serverless functions.	Design of Ratchet, a resource-adaptive query execution framework supporting query suspension, resumption, resource arbitration, and fine-grained pricing models.	Adaptive query suspension and resumption allow queries to tolerate transient resource loss while maintaining performance and cost efficiency under volatile cloud environments.
Wagner et al. (2024) [10]	Vectorized execution and JIT compilation each perform well under different workloads, but no single execution paradigm dominates across all scenarios.	Development of Incremental Fusion, a hybrid execution engine that dynamically switches between vectorized interpretation and operator-fusing code generation at runtime.	Incremental Fusion achieves competitive performance for both short and long-running queries, combining low-latency execution with high computational efficiency.
Bruno et al. (2024) [12]	Traditional query optimizers are inadequate for cloud-native data warehouses with disaggregated compute and storage.	Redesign of the Fabric Data Warehouse optimizer using hierarchical distributed planning, including global plan generation and local fragment optimization.	Cloud-aware hierarchical optimization significantly reduces data movement and produces efficient end-to-end distributed execution plans.
Chen et al. (2025) [13]	Existing HTAP systems struggle to balance OLTP and OLAP workloads under dynamic resource conditions.	Design of veDB-HTAP using MySQL Secondary Engine with adaptive query execution, runtime statistics collection, and cost-based and ML-based query routing.	Adaptive execution and intelligent routing improve performance and quality-of-service for mixed OLTP and OLAP workloads under heavy load.
Cao et al. (2022) [14]	Scaling relational databases elastically in cloud environments while maintaining performance remains challenging.	Design of PolarDB-X, a cloud-native distributed relational database with CN/DN architecture, MPP execution, and near-data computing, evaluated experimentally.	PolarDB-X demonstrates elastic scaling, efficient MPP query execution, and reduced data movement through operator pushdown to storage nodes.
Cao et al. (2021) [15]	Monolithic database architectures limit elasticity and efficient resource utilization in cloud data centers.	Design of PolarDB Serverless with fully disaggregated compute, memory, and storage pools supporting on-demand provisioning and independent scaling.	Disaggregated serverless architecture enables rapid scaling, elastic throughput, and faster recovery under fluctuating workloads.
Bang et al. (2021) [16]	Fixed database architectures restrict elasticity, adaptability, and heterogeneous hardware utilization.	Proposal of AnyDB, an architecture-less DBMS where data and control flow as event streams, enabling flexible event routing and heterogeneous execution.	Event-driven execution enables transparent elasticity, overlapping compilation and execution, and flexible use of heterogeneous compute resources.
Perron et al. (2020) [17]	Provisioned database clusters are cost-inefficient for sporadic or bursty analytical workloads.	Design of Starling, a serverless query engine that executes queries using cloud functions with intermediate results materialized in cloud storage.	Starling achieves low-cost elasticity and interactive latency by scaling function invocations on demand and charging users per query execution.

D. AI-Driven Query Optimization			
Ye et al. (2024) [23]	Traditional cost-based and learning-based query optimizers fail to exploit semantic equivalence among queries.	They proposed Celo, which is a constraint-enhanced learned query optimizer equivalent query.	They found significant improvements in query latency and execution cost.
Karri (2021) [24]	Classical cost-based optimizers struggle with inaccurate cardinality estimation and dynamic workloads.	Create a hybrid AI-assisted optimization framework that integrates learned cardinality estimation, neural cost correction, and reinforcement learning-based planning into an existing cost-based optimizer.	They found the AI can effectively augment traditional optimizers without replacing them.
Venkata (2025) [25]	Query optimization in multitenant database systems is challenging due to resource contention, workload diversity, and varying service-level requirements among tenants.	They applied AI-driven tenant-aware optimization using reinforcement learning, neural networks for cardinality estimation, and workload classification.	The results found improved performance isolation, better resource utilization, and reduced administrative overhead.
Sassi and Jaziri (2025) [26]	Traditional cost-based join order optimization performs poorly in large-scale and complex databases due to inaccurate cardinality estimates and limited exploration of plan space.	Proposed a hybrid AI-based framework combining Graph Neural Networks (GNNs) to model database schemas as graphs. Besides that, Reinforcement Learning (RL) use to optimize join order selection, trained using Proximal Policy Optimization (PPO).	Achieved substantial reductions in query execution time, CPU and memory usage, and cardinality estimation error compared to both traditional cost-based and existing learning-based optimizers.
Sun and Sun (2024) [27]	Real-time big data query processing systems suffer from inefficient task scheduling, leading to increased query latency and system cost.	Optimized task scheduling for real-time query processing using machine learning techniques	Experimental results showed notable reductions in query processing time and overall system cost

F. Literature Review Findings, Gaps, Opportunities

The literature reviews have found that improvements in DBMS query performance are achieved through the improve process at different stages. Pipeline-oriented scheduling help to reduce peak memory usage by control the execution order of operators and limit the size of intermediate results [3]. In addition, modern execution engines increasingly combine vectorized execution with compilation techniques and runtime profiling to achieve a balance between low latency and high throughput when handling mixed workloads [4][10][11].

Besides that, join processing has also made significant progress on improving robustness and scalability. For instance, multi-GPU sort-merge join techniques and skew-resistant hybrid hash join designs both show significant performance improvements for large-scale join workloads [5][6]. Furthermore, scan reduction techniques like hierarchical indexing and correlation-aware data layouts help reduce unnecessary I/O by skipping irrelevant data blocks, which lead to improve efficiency when processing large datasets [7][8]. At the system level, distributed and cloud-native databases introduce adaptive execution frameworks that support elasticity, pre-emption, and fault-tolerant query suspension to cope with volatile computing resources while maintaining acceptable performance and cost efficiency [9][12]–[17].

The literature also highlights the importance of join order optimization, where suboptimal join ordering can significantly increase intermediate result sizes and overall execution cost [18]. Besides that, cost-based optimization widely used in heterogeneous or multistore environments where execution plans must account for different engines and global cost trade-offs [19]. Furthermore, rule-based optimization and query rewriting techniques are used to simplify logical query structures to prevent rewriting explosion and excessive optimization overhead [20][21]. Nevertheless, survey studies point out that query optimizers are still limited by inaccurate cardinality estimation and incomplete plan enumeration, which can lead to suboptimal execution plans [22]. More recent studies explore AI-driven optimization approaches, including learned cardinality estimation and reinforcement learning-based planning, which have shown improvements in execution latency, plan stability, and resource utilization under complex workloads [23]–[27].

There are also some gaps remain. Firstly, many hybrid execution engines that combine vectorization and compilation will improve adaptability. However, they will increase the system complexity and require careful runtime decision-making to avoid overheads that offset performance gain [10][11]. Besides that, the AI-driven optimizer having challenges in training cost as it takes time and cost to train a model.

Thus, the future research opportunities would be designing a lightweight and adaptive hybrid execution engines that minimize the system complexity. Moreover, future research may find the more cost-efficient AI-driven query optimization method that can integrate with the traditional technique. These will help to reduce training time and computational cost while still benefiting from AI-assisted optimization.

G. Summary of Literature Review

In summary, the literature shows the significant improvements in DBMS query performance can be achieved using query processing and optimization techniques. While the effectiveness traditional optimization methods such as join order optimization, cost-based planning, and query rewriting remain fundamental but often limited by system complexity and inaccurate cardinality estimation. Furthermore, AI-driven optimization approaches provide better performance gains under complex workloads. However, it faces challenges related to training cost and practical deployment. Overall,

these studies indicate that future research should focus on balancing adaptability, efficiency, and complexity to achieve robust and scalable query optimization in modern database systems.

III. METHODOLOGY

A. Introduction to the Methodology

A mixed methodology was selected to allow both conceptual understanding and practical evaluation. The literature review provides the theoretical insights into query processing and optimization techniques. At the same time, SQL-based experimentation enables observation of their performance impact in a real DBMS environment.

B. Research Design

In this paper, we have reviewed the relevant academic literature related to query processing and query optimization based on their relevance to query execution pipelines, execution models, join processing, scan techniques, distributed query execution, and AI-driven optimization.

C. Step Involved in the Methodology

1) Identification of Research Methodology

A mixed methodology was selected to allow both conceptual understanding and practical evaluation. The literature review provides the theoretical insights into query processing and optimization techniques. At the same time, SQL-based experimentation enables observation of their performance impact in a real DBMS environment.

2) Literature Review and Analysis

We have reviewed the relevant academic literature related to query processing and query optimization based on their relevance to query execution pipelines, execution models, join processing, scan techniques, distributed query execution, and AI-driven optimization.

Each selected study was analysed in terms of:

- the problem addressed,
- the proposed techniques,
- experimental evaluation methods, and
- reported performance improvements.

The findings were synthesised to identify common optimization strategies, limitations of traditional approaches and emerging trends such as adaptive execution and AI-driven optimization. These insights informed the design of the experimental evaluation and the selection of optimization techniques tested in this study

3) Criteria for Comparison

To evaluate the effectiveness of query processing and optimization techniques, the following criteria were used:

- ***Query Execution Performance***

To measured using execution time (milliseconds) obtained from query execution statistics.

- ***Resource Efficiency***

Inferred from execution plans, including the number of documents or rows examined, index keys examined, and access paths (e.g., collection scan versus index scan).

- ***Optimization Effectiveness***

Evaluated by comparing query performance before and after applying optimization techniques such as indexing and query restructuring.

- ***Scalability Characteristics***

Assessed qualitatively by observing how queries behave when processing large datasets and complex aggregation operations.

4) Data Collection

The dataset used is the Online Retail dataset obtained from the UCI Machine Learning Repository. The dataset has included 541,909 transactional records of a UK-based online retail company. Each record contains attributes such as invoice number, product identifier, quantity, unit price, invoice date, customer identifier, and country.

The dataset represents a realistic large-scale transactional workload and is suitable for evaluating aggregation-heavy queries, customer behaviour analysis, and query optimization techniques. The data was imported into both MySQL and MongoDB to support comparative experimentation across relational and NoSQL environments.

5) Experimental Setup

The experiments were carried out on two laptops with the following specifications:

TABLE II. LAPTOP SPECIFICATIONS

Software	MySQL	MongoDB
Operating System	Windows 11 Home Single Language	
Processor	AMD Ryzen 7 7730U with Radeon Graphics	Intel Core i7-11800H @ 2.30GHz
Main Memory	8 GB RAM	16 GB RAM
Hard Disk	512 GB SSD	1TB SSD

6) Implementation of Solutions

The experimental evaluation was conducted using both MySQL and MongoDB environments.

In MongoDB, queries were implemented using the aggregation pipeline and index optimization. In MySQL, equivalent SQL queries were executed using GROUP BY, filtering predicates, and aggregation functions.

Two main optimization techniques were applied:

- **Indexing**

Single-field and compound indexes were created on frequently queried attributes such as invoice date, customer identifier, and country.

- **Query Restructuring**

Queries were rewritten to push filtering conditions as early as possible and to reduce unnecessary intermediate results. For each query, experiments were conducted before and after optimization, while keeping the query logic identical. Execution plans were analysed using DBMS-provided tools such as EXPLAIN and executionStats to understand optimizer decisions.

7) Evaluation and Comparison

Query performance was evaluated by comparing execution statistics before and after optimization. Metrics such as execution time, number of documents examined, and index usage were collected and analysed.

The results were compared across different query types to identify:

- scenarios where indexing significantly improved performance,
- cases where aggregation overhead dominated execution time, and
- situations where the query optimizer preferred collection scans due to low predicate selectivity.

This comparison highlights the conditions under which query optimization techniques are effective and when they provide limited benefit.

8) Validation of Results

To ensure reliability, each experiment was executed multiple times and average execution statistics were considered. Execution plans were cross-checked to confirm whether indexes were actually used by the optimizer.

Additionally, the observed results were validated against findings from the literature, particularly regarding the impact of predicate selectivity and aggregation complexity on optimization effectiveness. This triangulation strengthens the validity of the experimental findings.

D. Tools and Technologies Used

The following tools and technologies were used in this study:

- **MySQL:** Used to perform relational query processing and optimization experiments.
- **MongoDB:** Used to evaluate NoSQL query processing and optimization experiments.
- **UCI Online Retail Dataset:** Used as the experimental dataset.

E. Flow Diagram

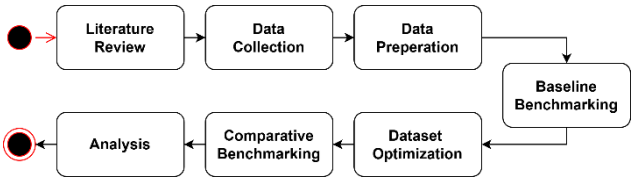


Fig. 1. Flow Diagram

Research & Data Collection:
The literature review is utilized to establish optimization techniques and procurement of UCI Online Retail data of approximately 541,000 records.

Data Preparation:
The raw CSV data will be imported into both the MySQL and the MongoDB environments and will be processed with the standard cleaning and type conversion to provide the appropriate accuracy.

Baseline Benchmarking:
The pre-optimized database is then run through a sequence of analysis queries which document the baseline execution time and resource utilization.

Optimization & Comparative Testing:
Optimization strategies are then implemented. The same queries are then re-executed to measure performance gains.

Analysis:
Finally, the results are analysed to quantify the improvements in latency and efficiency before concluding the study.

F. Challenges and Limitation

1. Data Type Heterogeneity and Schema Enforcement

A significant challenge was caused by the fundamental different in MySQL and MongoDB as MySQL imposed rigid data types when creating a table while MongoDB is not schema based. Therefore, all attributes that were initially imported are settings as a string. This required an additional post-processing phase with the use of the $toInt, $toDouble and $dateFromString operators to put 541,000 documents in useful number and date formats before any analysis could occur, this causes an additional computational cost.

2. Handling Dirty Data

The InvoiceDate had both mixed text types of 12/1/2010 and 01/12/2010 which caused standard import wizards to fail. This necessitated a custom ETL (Extract, Transform, Load) script to edit the dates and convert them into a standard DATETIME format ensure that the time-series analysis was done correctly.

3. Complexity of NoSQL

Unlike MySQL's declarative syntax such as Group By, MongoDB required the construction of imperative pipelines such as $match, $\rightarrow$, $group,

$\rightarrow$ and more. As the lack of experience using NoSQL, these multi-stage pipelines were more difficult to debug, especially with such calculated fields as Revenue (Quantity x Unit price) that needed the use of $multiply operator in the projection phase instead of a simple arithmetic expression.

G. Summary

This chapter adopts a mixed-method experimental approach to evaluate query optimization in relational and non-relational paradigms. The study utilized the UCI Online Retail dataset that used to compare MySQL 8.0 and MongoDB under operation conditions. The methodology included three critical phases which are Data Pre-processing which involve SQL transformation scripts and MongoDB type-conversion pipelines, Baseline Benchmarking of read-heavy analytical queries, and implementation optimization strategies to improve the operational time needed.

A note to mention is that the experimental design acknowledges the architectural differences between the two systems. Whereas the MySQL experiments were mainly based on index seek latency to find a particular record through a B-Tree query, the MongoDB experiments were based on the efficiency of the aggregation pipeline to calculate complex summaries. Therefore, the findings in the next chapter indicate two different aspects of performance MySQL is more efficient at transactional retrieval as compared to MongoDB which is more efficient at processing documents analytically.

IV. Results and Discussion

A. MySQL

1. Data Setting

Figures 2 and 3 collectively illustrate the data ingestion pipeline, starting with the initialization of the raw_online_retail staging table where loose data types like VARCHAR are used to prevent format-related errors during the initial creation. Once the structure is established, the process enables local file access permissions and executes the LOAD DATA command to efficiently bulk transfer the raw CSV records into the database, automatically handling field termination and header skipping.

Meanwhile, Table 3 summarises the characteristics of each attribute's data type when creating the table.

TABLE III. Data Setting for MySQL

Attribute	Data Type
InvoiceNo	Varchar
StockCode	Varchar
Description	Varchar
Quantity	Int
InvoiceDate	Varchar
UnitPrice	Double (Floating)
CustomerID	Int
Country	Varchar

```sql
USE onlineretail;

DROP TABLE IF EXISTS raw_online_retail;
CREATE TABLE raw_online_retail (
    InvoiceNo VARCHAR(20),
    StockCode VARCHAR(20),
    Description VARCHAR(255),
    Quantity INT,
    InvoiceDate VARCHAR(30),
    UnitPrice DECIMAL(10,2),
    CustomerID INT,
    Country VARCHAR(50)
);
```

Fig. 2. Table Creation Script

```sql
SET GLOBAL local_infile = 1;

LOAD DATA LOCAL INFILE
'C:/Sunway University/DATABASE MANAGEMENT SYSTEMS/Assignment/retail.csv'
INTO TABLE raw_online_retail
CHARACTER SET latin1
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country);
```

Fig. 3. Data Import Script

2. Data Cleaning (Optimization Level 1)

After loading the raw data, the column of InvoiceDate which currently is text-based, needs to be transformed into a correct format of DATETIME in order to allow effective time-based filtering and helping indices. The following script will fix the mismatched date strings and afterwards change table schema to make the typed DATETYPE permanent.

```sql
SET SQL_SAFE_UPDATES = 0;

UPDATE raw_online_retail
SET InvoiceDate =
    CASE
        WHEN SUBSTRING_INDEX(InvoiceDate, '/', 1) > 12
        THEN STR_TO_DATE(InvoiceDate, '%d/%m/%Y %H:%i')
        ELSE STR_TO_DATE(InvoiceDate, '%m/%d/%Y %H:%i')
    END;

ALTER TABLE raw_online_retail
MODIFY InvoiceDate DATETIME;
```

Fig. 4. Data Cleaning and Type Conversion

3. Before Optimization vs After Optimization

In this section, the performance of the selected analytical queries on before and after optimization with regard to execution are displayed. Execution time, number of documents examined and number of index keys examined are used to determine performance. The comparison brings out instances where optimization worked and instances where there was no change in performance.

a. *Scenario 1: Monthly Transaction Volume (Date Range Scan)*

A retail business setting is a highly competitive environment and verifying the amount of sales made within a given time frame is significant important daily task. This scenario examine the performance of counting of all the transactions that have taken place in December 2010. The date filtering is important to be efficient in that the database server is not overloaded to produce a timely monthly report.

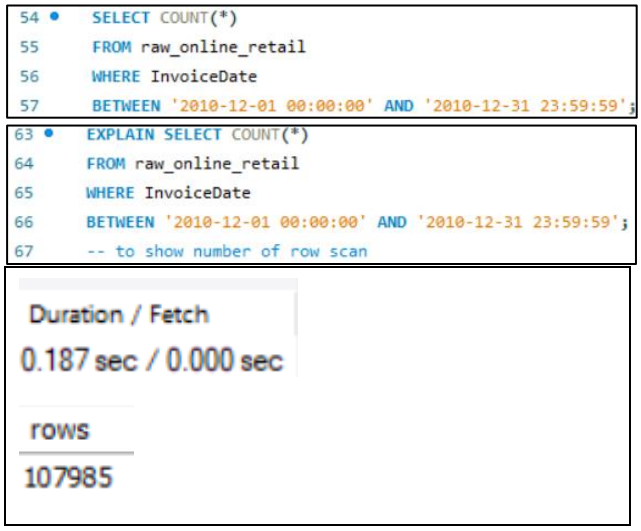


Fig. 5. (Before Optimization) Execution Statistics

Before optimization, figure 5 shows that the execution statistics show that MySQL examined 107985 rows. This indicates that the database performed an inefficient scan, using 187ms to perform this query.

```
63 •  CREATE INDEX idx_invoice_date
64    ON raw_online_retail(InvoiceDate);
```

Fig. 6. Index Optimization

An index named *idx_invoice_date* was created on the *InvoiceDate* attribute. This allows the database engine to perform a Range Scan, jumping directly to the start of the date range rather than scanning the entire table.

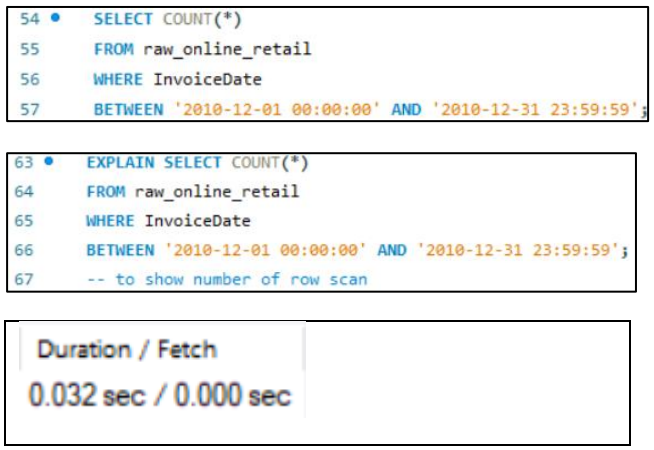




Fig. 7. (After Optimization) Execution Statistics

After optimization, figure 7 shows that the execution statistics show that MySQL significantly improve the from 107985 to 31848 rows. The query optimizer successfully utilized the index to locate the relevant records immediately. It used 16ms to perform this query.

TABLE IV. COMPARISON TABLE

Query	Execution Time (ms)	Total Row Examined
Monthly Transaction Volume (Before)	187	107985
Monthly Transaction Volume (After)	32	31848

b. *Scenario 2: Customer Activity Lookup (Point Lookup)*

Customer service representatives frequently need to retrieve full transaction histories for specific customers to resolve complaints or verify orders. This scenario analyses the retrieval of all records for a specific *CustomerID*.

```
67 •  SELECT * FROM raw_online_retail
68    WHERE StockCode = '85133A';
```

```
75 •  Explain SELECT * FROM raw_online_retail
76    WHERE StockCode = '85133A';
77    -- to show number of row scan
```

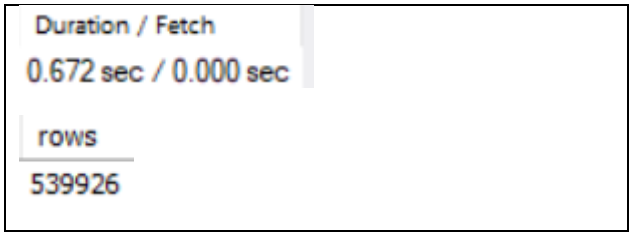


Fig. 8. (Before Optimization) Execution Statistics

Before optimization, figure 8 shows that the execution statistics show that MySQL examined the entire dataset of 539926 rows. This indicates that the database performed a Full Table Scan, reading every record in the collection to check the date range. It used 672ms to perform this query.

```
72 •  CREATE INDEX idx_stock_code
73    ON raw_online_retail(StockCode);
```

Fig. 9. Index Optimization

An index was added on StockCode attribute titled as idx_stock_code. This enables the database engine to do a Point Lookup whereby to instantly locate the specific key value.

```
67 •  SELECT * FROM raw_online_retail
68    WHERE StockCode = '85133A';
```

```
75 •  Explain SELECT * FROM raw_online_retail
76    WHERE StockCode = '85133A';
77    -- to show number of row scan
```

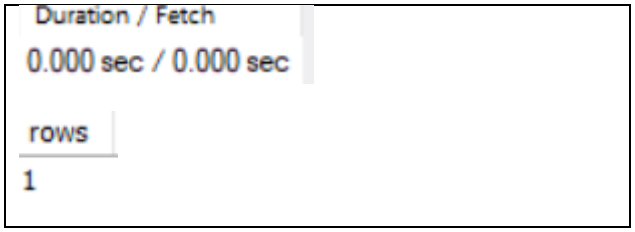


Fig. 10. (After Optimization) Execution Statistics

Figure 10 shows that the query cost reduced significantly after optimization. The execution statistics show that MySQL examined only 1 row which mean MySQL go directly to the exact location of the data. This query was done by using <1ms.

TABLE V. COMPARISON TABLE

Query	Execution Time (ms)	Total Row Examined
Customer Activity Lookup (Before)	672	539926
Customer Activity Lookup (After)	<1ms	1

c. *Scenario 3: High-Performance Aggregation (Covering Index)*

Total spending per customer is an heavy task when frequent reporting dashboards are calculated. This scenario applies to a Covering Index method where the index itself holds all the information needed to complete the query such as CustomerID, Unitprice, Quantity, allowing the database to skip reading the table.

```
123 •  SELECT CustomerID, SUM(UnitPrice * Quantity)
124    FROM raw_online_retail
125    GROUP BY CustomerID;
```

```
154 •  explain SELECT CustomerID, SUM(UnitPrice * Quantity)
155    FROM raw_online_retail
156    GROUP BY CustomerID;
157    -- to show number of row scan
```

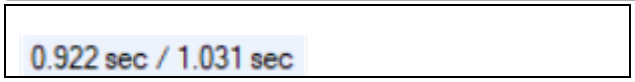




Fig. 11. (Before Optimization) Execution Statistics

Before optimization, figure 11 show that the execution statistics show that MySQL examined the entire dataset of 539926 rows. This indicates that the database performed a Full Table Scan, reading every record in the collection to calculate the sum for each customer. It used 922ms to perform this query.

```
139 •  CREATE INDEX idx_money_covering
140    ON raw_online_retail(CustomerID, UnitPrice, Quantity);
```

Fig. 12. Index Optimization

A composite index idx_money_covering was developed with all the three required columns. This allows the database to jumping directly to the start of the date range rather than scanning the entire table.

```
123 •  SELECT CustomerID, SUM(UnitPrice * Quantity)
124    FROM raw_online_retail
125    GROUP BY CustomerID;
```

```
154 •  explain SELECT CustomerID, SUM(UnitPrice * Quantity)
155    FROM raw_online_retail
156    GROUP BY CustomerID;
157    -- to show number of row scan
```

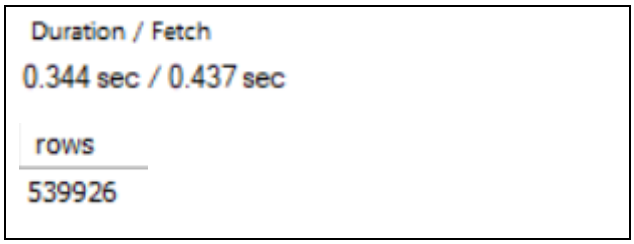


Fig. 13. After Optimization

After optimization, figure 13 show the execution time improve significantly to 344ms, while the row count remain the same at 539926 as the database need to read every record to calculate the total sum. The database utilized the pre-sorted index to perform the calculation more efficiently.

TABLE VI. COMPARISON TABLE

Query	Execution Time (ms)	Total Row Examined
High-Performance Aggregation (Before)	922	539926
High-Performance Aggregation (After)	344	539926

d. Scenario 4: Managing Massive Historical Data (Table Partitioning)

With the development of the retail dataset over a few years, the size of the table may become bulky, and the maintenance operations such as removal of old data or backup of a particular year will become excruciatingly slow. This situation adopts Range Partitioning, which is a method of physically dividing the table to smaller manageable sub-tables depending on the transaction year.

```
104 • CREATE TABLE retail_partitioned (
105       InvoiceNo VARCHAR(20),
106       StockCode VARCHAR(20),
107       Description VARCHAR(255),
108       Quantity INT,
109       InvoiceDate DATETIME,
110       UnitPrice DECIMAL(10,2),
111       CustomerID INT,
112       Country VARCHAR(50)
113   )
114   PARTITION BY RANGE ( YEAR(InvoiceDate) ) (
115       PARTITION p2010 VALUES LESS THAN (2011),
116       PARTITION p2011 VALUES LESS THAN (2012),
117       PARTITION p_future VALUES LESS THAN MAXVALUE
118   );
```

Fig. 14. Creating Partitioned Table Structure

A new table structure retail_partitioned was developed instead of using queries on a gigantic table. This table is partitioned according to the InvoiceDate. For example, p2010, p2011.

```
126 •  INSERT INTO retail_partitioned
127    SELECT * FROM raw_online_retail;
```

Fig. 15. Data Migration Script

Figure 15 show the data migration after setting up the partitioned structure, the existing data in the staging table (raw_online_retail) was migrated to the new partitioned table by means of an INSERT INTO ... SELECT statement. This automatically partitions the rows into their respective year-partitions.

```
134 •  SELECT COUNT(*) FROM raw_online_retail
135    WHERE InvoiceDate
136    BETWEEN '2011-12-01' AND '2011-12-31';
```

```
95 •  Explain SELECT COUNT(*) FROM raw_online_retail
96    WHERE InvoiceDate
97    BETWEEN '2011-12-01' AND '2011-12-31';
98    -- to show number of row scan
```

Duration / Fetch
0.109 sec / 0.000 sec

rows
23326

Fig. 16. Before Optimization

Before optimization, Figure 16 demonstrates that the query took 0.109 seconds and 23326 rows were searched. This high-performing query is possible since the query made use of the B-Tree index of InvoiceDate (created in Scenario 1), and thus the database was able to select the exact records instantly.

```
130 •  SELECT COUNT(*) FROM retail_partitioned
131    WHERE InvoiceDate
132    BETWEEN '2011-12-01' AND '2011-12-31';
```

```
124 •  Explain SELECT COUNT(*) FROM retail_partitioned
125    WHERE InvoiceDate
126    BETWEEN '2011-12-01' AND '2011-12-31';
```

Duration / Fetch
0.297 sec / 0.000 sec

rows
496641

Fig. 17. (After Optimization) Execution Statistics

After optimization, figure 17 demonstrates that the query took 0.297 seconds and processed 496641 rows. The database was still required to scan the whole partition that it had to find the records even though the table was partitioned. It proves that at such large datasets that around 500k rows, an accurate Index look can be quicker than Partitioning because the cost of scanning such a large partition is greater than the advantage of pruning the 2010 data.

TABLE VII. COMPARISON TABLE

Query	Execution Time (ms)	Total Row Examined
High Performance Aggregation (Before)	109	23326
High Performance Aggregation (After)	297	496641

B. MongoDB

1. Data Setting

Figure 18 illustrates the data preparation process applied to the transactions collection after importing the dataset from CSV format. As CSV import stores all values as strings, key attributes were standardised into appropriate data types to ensure correct query execution. Numerical fields such as *Quantity* and *UnitPrice* were converted to numeric types. *CustomerID* was normalised to handle missing values and *InvoiceDate* was converted into a date format to support time-based filtering. In addition, a new computed attribute, Revenue was derived from the product of quantity and unit price to simplify aggregation queries (Figure 19).

Meanwhile, Table 5 summarises the characteristics of each attribute's data type after standardisation, while the figures present the corresponding implementation code used for data type conversion and revenue attribute creation. This data preparation step ensures accurate query processing and reliable performance evaluation in subsequent experiments

TABLE VIII. DATA SETTING FOR MONGODB

Attribute	Data Type
InvoiceNo	String
StockCode	String
Description	String
Quantity	Int32
InvoiceDate	Date
UnitPrice	Double (Floating)
CustomerID	Int32
Country	String
Revenue	Double (Floating)

```
Retail> db.transactions.updateMany(
        {},
        [
          {
            $set: {
              Quantity: { $toInt: "$Quantity" },
              UnitPrice: { $toDouble: "$UnitPrice" },
              CustomerID: {
                $cond: [
                  { $or: [{ $eq: ["$CustomerID", ""] }, { $eq: ["$CustomerID", null] }] },
                  null,
                  { $toInt: "$CustomerID" }
                ]
              },
              InvoiceDate: {
                $dateFromString: {
                  dateString: "$InvoiceDate",
                  format: "%m/%d/%Y %H:%M"
```

Fig. 18. Data Type Setting

```
Retail> db.transactions.updateMany(
        {},
        [{ $set: { Revenue: { $multiply: ["$Quantity", "$UnitPrice"] } } }]
      )
```

Fig. 19. Revenue attributes

2. Before Optimization vs After Optimization

This section shows the execution performance of selected queries before and after optimization. The performance is evaluated based on using execution time, number of documents examined, and number of index keys examined provided by the system feedback.

a) Scenario 1: Revenue by Country

In order to identify key markets and support strategic decision-making, a business need to understand the revenue across different countries. Based on the past data, they will examine the overall revenue produced by each country. The query aggregates sales data across the dataset to get revenue contributions for all countries. This type of query represents a common high-level analytical workload used in business performance reporting. Figure 20 shows the query code that explain the query time to obtain revenue by country information.

```
Retail> db.transactions.explain("executionStats").aggregate([
          { $match: { Quantity: { $gt: 0 }, UnitPrice: { $gt: 0 } } },
          { $group: { _id: "$Country", totalRevenue: { $sum: "$Revenue" } } },
          { $sort: { totalRevenue: -1 } },
          { $limit: 10 }
        ])
```

Fig. 20. Query code

```
    executionTimeMillis: 486,
    totalKeysExamined: 0,
    totalDocsExamined: 541909,
    executionStages: {
      stage: 'project',
      planNodeId: 3,
      nReturned: 38,
      executionTimeMillisEstimate: 475,
      opens: 1,
      closes: 1,
      saveState: 32,
      restoreState: 31,
      isEOF: 1,
      projections: {
        '11': 'newObj("_id", s8, "totalRevenue", s10) '
      },
```

Fig. 21. (Before Optimization) Execution Statistics

Before optimization, the execution statistics show that MongoDB examined 541909 documents and 0 index keys. This indicates that MongoDB performed a collection scan by reading every document in the collection. It used 475ms to perform this query (Figure 21).

```
> db.transactions.createIndex({ Country: 1 })
< Country_1
```

Fig. 22. Index Optimization

Figure 22 shows the index created on the *Country* attribute to allowing MongoDB to organise records based on country. This index could help the database quickly locate records belonging to specific countries during query execution.

```
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 38,
    executionTimeMillis: 529,
    totalKeysExamined: 0,
    totalDocsExamined: 541909,
    executionStages: {
      stage: 'project',
      planNodeId: 3,
      nReturned: 38,
      executionTimeMillisEstimate: 523,
      opens: 1,
      closes: 1,
      saveState: 36,
      restoreState: 35,
      isEOF: 1,
      projections: {
        '11': 'newObj("_id", s8, "totalRevenue", s10) '
```

Fig. 23. (After Optimization) Execution Statistics

After optimization, figure 23 show the execution statistics where MongoDB examined 541909 documents

and 0 index keys. This indicates that MongoDB have read every document in the collection and did not use the index. It used 529ms to perform this query.

TABLE IX. COMPARISON TABLE

Query	Execution Time (ms)	Total Docs Examined	Total Key Examined
Revenue by Country (Before)	486	541909	0
Revenue by Country (After)	529	541909	0

The comparison table above (Table 9) show that the query did not getting optimized from index optimization. MongoDB examined the entire dataset and did not use any index where zero index keys examined. This issue happens because the query aggregates revenue across all countries and there are no filtering conditions. This query request has match almost all records in the dataset which cause the query optimizer to select a collection scan as the index usage do not help to reduce the execution cost and time.

b) Scenario 2: Finding Specific Data

The analyst needs to retrieve specific subsets of data based on certain conditions to do analytic and prediction. This scenario aims to extract a targeted subset of records from the dataset using selective conditions. By reduce the search range, the query optimization can potentially reduce execution cost and improve performance. In this testing, figure 24 have match the revenue ranging from 1 November 2011 to 1 December 2011 that made in United Kingdom



Fig. 24. Query Code



Fig. 25. (Before Optimization) Execution Statistics

Before optimization, figure 25 show the execution statistics where MongoDB examined 541909 documents and 0 index keys. This indicates that MongoDB performed a collection scan by reading every document in the collection. It used 307ms to perform this query.

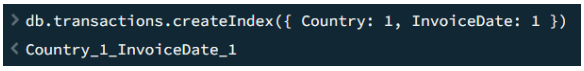


Fig. 26. Index Optimization

Figure 26 show the index created on *Country* and *InvoiceDate*. This index is designed to support queries that will required filter geographic location and time range. In this case, it enable faster access to a specific subset of data.



Fig. 27. (After Optimization) Execution Statistics

After optimization, figure 27 show the execution statistics where MongoDB examined 78640 documents and 78640 index keys. This indicates that MongoDB used the index to locate relevant records matching the filtering conditions. It accessed only indexed entries within the specified country and date range. The execution time reflects the reduced workload caused by limiting the number of documents processed.

TABLE X. COMPARISON TABLE

Query	Execution Time (ms)	Total Docs Examined	Total Key Examined
Finding Specific Data (Before)	307	541909	0
Finding Specific Data (After)	126	78640	78640

The comparison table (Table 10) above shows the execution statistics of the specific finding query before and after index optimization. Before optimization, it examined the entire dataset and did not utilise any index. After index optimization, the number of documents examined was reduced to 78640 and an equivalent number of index keys. This proof that the index was used during query execution. As a result, the execution time decreased from 307ms to 126ms. This indicates that index optimization was effective in reducing the query execution cost by limiting the number of documents processed.

c) Scenario 3: Top Selling Product in a Specific Month

Retail companies often analyse product-level sales performance within specific time periods to identify best-selling items and support inventory planning. This scenario focuses on identifying the top-selling products during a particular month by aggregating transaction quantities. Figure 28 shows the query code to find the top selling product during 1 November 2011 and 1 December 2011.

```
Retail> db.transactions.explain("executionStats").aggregate(
        {
          $match: {
            InvoiceDate: {
              $gte: ISODate("2011-11-01"),
              $lt: ISODate("2011-12-01")
            },
            Quantity: { $gt: 0 }
          }
        },
        {
          $group: {
            _id: "$StockCode",
            totalQuantity: { $sum: "$Quantity" }
          }
        },
        { $sort: { totalQuantity: -1 } },
        { $limit: 10 }
      ])
```

Fig. 28. Query Code

```
executionStats: {
  executionSuccess: true,
  nReturned: 2946,
  executionTimeMillis: 349,
  totalKeysExamined: 0,
  totalDocsExamined: 541909,
  executionStages: {
    stage: 'project',
    planNodeId: 3,
    nReturned: 2946,
    executionTimeMillisEstimate: 332,
    opens: 1,
    closes: 1,
    saveState: 27,
    restoreState: 26,
    isEOF: 1,
    projections: {
      '10': 'newObj("_id", s7, "totalQuantity", s9) '
```

Fig. 29. (Before Optimization) Execution Statistics

Before optimization, figure 29 show the execution statistics where MongoDB examined 541909 documents and 0 index keys. This indicates that MongoDB performed a collection scan by reading every document in the collection. It used 349ms to perform this query.

```
> db.transactions.createIndex({ InvoiceDate: 1, StockCode: 1 })
< InvoiceDate_1_StockCode_1
```

Fig. 30. Index Optimization

Figure 30 show the compound index was created on the attributes InvoiceDate and StockCode to support time-based product analysis queries. The index is ordered primarily by InvoiceDate, which allow MongoDB to efficiently locate records within a specific time range.

```
executionStats: {
  executionSuccess: true,
  nReturned: 2946,
  executionTimeMillis: 158,
  totalKeysExamined: 84711,
  totalDocsExamined: 84711,
  executionStages: {
    stage: 'project',
    planNodeId: 4,
    nReturned: 2946,
    executionTimeMillisEstimate: 142,
    opens: 1,
    closes: 1,
    saveState: 17,
    restoreState: 16,
    isEOF: 1,
    projections: {
      '14': 'newObj("_id", s11, "totalQuantity", s13) '
```

Fig. 31. (After Optimization) Execution Statistics

After optimization, figure 31 show the execution statistics where MongoDB examined 84711 documents and 84711 index keys. This indicates that MongoDB used the index to locate relevant records matching the filtering conditions. By using the compound index, it directly locate records that satisfy the filtering conditions. By using index, the query execution cost was reduced, resulting in a lower execution time.

TABLE XI. COMPARISON TABLE

Query	Execution Time (ms)	Total Docs Examined	Total Key Examined
Top Selling Product in a Specific Month (Before)	349	541909	0
Top Selling Product in a Specific Month (After)	158	84711	84711

The comparison table above (Table 11) shows the execution statistics before and after index optimization. Before optimization, it examined the entire dataset and did not utilise any index. After index optimization, the number of documents examined was reduced to 84711 with the same number of index keys. This indicates that the index on *InvoiceDate* and *StockCode* was successfully used to restrict the query to records within the specified month. As a result, the execution time decreased from 349ms to 158ms. This indicates that index optimization was effective in reducing the query execution cost.

3. Summary

This experiment evaluated the effectiveness of query processing and index optimization techniques. Three analytical scenarios were tested, which is revenue aggregation by country, selective data retrieval, and top-selling product analysis within a specific time period. Query performance was evaluated using execution time, number of documents examined, and number of index keys examined.

The results show that index optimization is highly dependent on query characteristics. In Scenario 1, index optimization was ineffective because the query aggregated data across the entire dataset without selective filtering. This led MongoDB to prefer a collection scan. Scenario 2 and Scenario 3 have a successful optimization process. The selective predicates such as country and date range significantly reduced the search space. In these cases, the query to have a lower execution time, which help to improve efficiency.

Overall, the findings found that index optimization is effective when queries apply selective conditions that reduce the number of records processed. For queries that process nearly all documents, index usage does not provide significant improvement. These have shown that query optimization will not always work for all query scenarios.

C. Limitation

1) Effectiveness of "Group-All" Optimization

The study shows that there was a functional limitation of optimizing a specific aggregation type. The “Group By Country” queries in which the aim is to summarize the whole dataset, indexing in MongoDB did not help by much. This represents a limitation in which the standard indexing techniques cannot be effective in those queries that are bound to perform complete collection scan regards of the database engine.

2) Cold Cache Variance

During the Benchmarking phase, results were displayed a "Cold Cache" variance, where initial query e executions were significantly slower because of the disk I/O latency. Although this was compensated by preloading the cache with frequently accessed content, it is still a weakness of testing on a non-dedicated server where background OS processes can affect disk access speeds.

The findings of the experiment showed that the indexing also acted differently in the two systems. Whereas the B-Tree indexing used in MySQL produced steadily growing returns, the MongoDB indexes produced uneven returns based on the type of query. As an example, aggregation queries with high cardinality (e.g., "Revenue by Country" did not improve performance after indexing, because the engine still had to scan a collection in order to aggregate documents (Source 301). This pointed to a shortcoming where aggregation tasks that require indexing which are 1:1 in SQL are not necessarily the same in NoSQL without indexing. especially with such calculated fields as Revenue (Quantity x Unit price) that needed the use of $multiply operator in the projection phase instead of a simple arithmetic expression.

D. Linking to Literature Review

The experimental findings in this study are consistent with the concepts and limitations discussed in the literature on query processing and optimization. The previous studies highlight that pipeline scheduling and execution model design play a critical role in reducing memory usage and balancing performance across workloads [3][4][10][11].In the experimental results, selective queries benefited from early filtering and index usage. Thus, this aligns with the literature’s emphasis on reducing intermediate results and improving execution efficiency through optimized execution pipelines.

Besides that, the observed behaviour of join and aggregation queries also reflects findings from previous research. Studies on join processing indicate that performance gains are highly dependent on data distribution and workload characteristics [5][6].In this study, aggregation queries that processed the entire dataset showed limited benefits from index optimization. This supports the literature’s view that scan-heavy analytical workloads require more than traditional indexing techniques. Similarly, scan reduction techniques discussed in the literature [7][8] explain why selective filtering conditions enabled MongoDB to effectively utilise indexes and reduce the number of documents examined.

Moreover, the findings are consistent with the studies involving distributed and cloud-native query execution frameworks. The observed problematic mix in the performance of index optimization in MongoDB indicates the issues outlined in the literature about the elasticity, resource volatility and unpredictability of execution behaviour in cloud-focused architectures [9][12]17]. This argument is supported by the experimental results indicating that the effectiveness of optimization is not limited to the design of indices but also depends on the models of execution and system architecture. Altogether, the findings confirm important findings in the literature review. In short, it shows that query optimization methods should be well tuned to query characteristics and the database system design.

E. Conclusion for Results and Discussion

This chapter compared the performance of query processing and optimization methods between the relational and NoSQL database systems on the MySQL and MongoDB platforms. The study presents how indexing, query restructure and data organisation on UCI Online Retails have influence in executing various query situations through a series of controlled experiments.

The findings indicate that index optimization is significantly efficient in selective queries such as date-range filtering, point lookups and target aggregations. In MySQL, the execution time and the number of rows accessed was highly decrease using B-tree indexes,

covering indexes and table partitioning. Whereas in MongoDB, compound indexes enhanced the aggregate pipeline by using selective filters at the beginning of the execution process. These results verify that optimization techniques are the beneficial when they lead to decreased data processed when executing queries.

Nevertheless, there were also limitations in the study. For example, aggregation-heavy queries with no selective predicates such as grouping across the dataset, indexing did not substantially help in MongoDB as the engine still need to do a complete collection scan. This highlights a fundamental limitation of the indexing strategies and the essence of the necessity to comprehend query semantics prior to the implementation of optimization strategies.

In general, the findings indicate that the effectiveness of query optimization depends on the context itself, tightly associated with query properties, data distribution, and system structure. Relational databases show a better performance in transaction or index-based access, whereas NoSQL systems perform better in flexible analytical model when assisted by selective filtering. These insights reinforce the necessity to pursue optimization mechanisms. The results of this chapter provide a strong foundation for the future research on the adaptive and AI-assisted query optimization methods that can dynamically adjust the implementation strategies in response to the workload and system restrictions.

V.

VI. Conclusion

This paper reviewed and analysed the literature available regarding query processing and optimization in database management systems, both in a traditional centralized database management system and the modern distributed and large-scale databases. The discussion summarized findings from various peer-reviewed articles to describe internal query execution using logical and physical execution plans, join processing strategies, and data access mechanisms including sequential and index scans. These underlying processes were demonstrated to be important determinants of query performance and the overall performance of the system.

The findings show that although the traditional query processing methods can be successfully applied to the single-node setting, when applied directly to the distributed and large-scale databases, they introduce new difficulties. Data partitioning strategies and parallel query execution strategies, query shipping and network communication overhead considerably influence performance and scalability. Overall, the studies reviewed support the idea that optimization choices in distributed systems must consider the trade-off between computation cost and communication cost, especially in the context of large datasets and their processing by multiple nodes.

Although significant improvements have been made in query processing and optimization several limitations remain. Most of the current methods are based on a set of static assumptions regarding the data distribution and the nature of workloads, which might not perform well in dynamic or heterogeneous environments. Moreover, network latency and synchronization overhead remain an issue in achieving optimal performance in distributed query execution.

Future studies may focus on more adaptive and scalable optimization methods that can more closely consider the variability of the workload and communication costs of large-scale systems. Besides, more efficient data partitioning schemes and better coordination mechanisms that ensure parallel execution can also be investigated further improve query performance. In general, this paper offers an organized view of contemporary methods of query processing and optimization and shows the implications of these methods the design and operation of modern database systems.

https://github.com/ItzSeir/G30_Database_Code

Reference

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database system concepts: Mcgraw-hill New York, 2002.

[2] S. Deochake, and D. Mukhopadhyay, “Cost-Aware Text-to-SQL: An Empirical Study of Cloud Compute Costs for LLM-Generated Queries,” arXiv preprint arXiv:2512.22364, 2025.

[3] L. Landgraf, F. Wolf, A. Boehm, and W. Lehner, Memory Efficient Scheduling of Query Pipeline Execution, Jan. 2022. doi:https://vldb.org/cidrdb/papers/2022/p82-landgraf.pdf#:~:text=will%20be%20consumed%20by%20subsequent,optimal

[4] F. Zhang et al., “AnalyticDB-PG: A cloud-native high-performance data warehouse in Alibaba Cloud,” Proceedings of the VLDB Endowment, vol. 18, no. 12, pp. 5139–5152, Aug. 2025. doi:10.14778/3750601.3750633

[5] S. Jahangiri, M. J. Carey, and J.-C. Freytag, “Design trade-offs for a robust dynamic hybrid hash join,” Proceedings of the VLDB Endowment, vol. 15, no. 10, pp. 2257–2269, Jun. 2022. doi:10.14778/3547305.3547327

[6] T. Maltenberger, I. Tolovski, and T. Rabl, “Efficiently joining large relations on Multi-GPU Systems,” Proceedings of the VLDB Endowment, vol. 18, no. 11, pp. 4653–4667, Jul. 2025. doi:10.14778/3749646.3749720

[7] M. Loporchio, A. Bernasconi, D. Di Francesco Maesa, and L. Ricci, “Skip index: Supporting efficient inter-block queries and query authentication on the Blockchain,” Future Generation Computer Systems, vol. 164, p. 107556, Mar. 2025. doi:10.1016/j.future.2024.107556

[8] S. Sudhir et al., “Pando: Enhanced Data Skipping with logical data partitioning,” Proceedings of the VLDB Endowment, vol. 16, no. 9, pp. 2316–2329, May 2023. doi:10.14778/3598581.3598601

[9] R. Liu et al., Towards Resource-adaptive Query Execution in Cloud Native Databases, Jan. 2024. doi:https://par.nsf.gov/servlets/purl/10524805

[10] B. Wagner, A. Kohn, P. Boncz, and V. Leis, “Incremental fusion: Unifying compiled and vectorized query execution,” 2024 IEEE 40th International Conference on Data Engineering (ICDE), pp. 462–474, May 2024. doi:10.1109/icde60146.2024.00042

[11] F. Schiavio, D. Bonetta, and W. Binder, “Dynamic Speculative Optimizations for SQL compilation in Apache Spark,” Proceedings of the VLDB Endowment, vol. 13, no. 5, pp. 754–767, Jan. 2020. doi:10.14778/3377369.3377382

[12] N. Bruno et al., “Unified query optimization in the Fabric Data Warehouse,” Companion of the 2024 International Conference on Management of Data, pp. 18–30, Jun. 2024. doi:10.1145/3626246.3653369

[13] J. Chen et al., “VeDB-HTAP: A highly integrated, efficient and adaptive HTAP system,” Proceedings of the VLDB Endowment, vol. 18, no. 12, pp. 4896–4909, Aug. 2025. doi:10.14778/3750601.3750614

[14] W. Cao et al., “PolarDB-X: An elastic distributed relational database for cloud-native applications,” 2022 IEEE 38th International Conference on Data Engineering (ICDE), pp. 2859–2872, May 2022. doi:10.1109/icde53745.2022.00259

[15] W. Cao et al., “PolarDB serverless,” Proceedings of the 2021 International Conference on Management of Data, pp. 2477–2489, Jun. 2021. doi:10.1145/3448016.3457560

[16] T. Bang, N. May, I. Petrov, and C. Binnig, AnyDB: An Architecture-less DBMS for AnyWorkload, Jan. 2021. doi:https://vldb.org/cidrdb/papers/2021/cidr2021_paper10.pdf

[17] M. Perron, R. Castro Fernandez, D. DeWitt, and S. Madden, “Starling: A scalable query engine on cloud functions,” Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, pp. 131–141, May 2020. doi:10.1145/3318464.3380609

[18] Y. Wu, J. M. Patel, and H. V. Jagadish, "Structural join order selection for XML query optimization." pp. 443-454.

[19] C. Forresi, M. Francia, E. Gallinucci, and M. Golfarelli, “Cost-based Optimization of Multistore Query Plans,” Information Systems Frontiers, vol. 25, no. 5, pp. 1925-1951, 2023/10/01, 2023.

[20] G. Gottlob, G. Orsi, and A. Pieris, “Query Rewriting and Optimization for Ontological Databases,” ACM Trans. Database Syst., vol. 39, no. 3, pp. Article 25, 2014.

[21] A. Alhazmi, T. Blount, and G. Konstantinidis, “ForBackBench: a benchmark for chasing vs. query-rewriting,” Proc. VLDB Endow., vol. 15, no. 8, pp. 1519–1532, 2022.

[22] H. Lan, Z. Bao, and Y. Peng, “A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration,” Data Science and Engineering, vol. 6, no. 1, pp. 86-101, 2021/03/01, 2021.

[23] C. Ye, H. Duan, H. Zhang, Y. Wu, and G. Dai, "Learned Query Optimization by Constraint-Based Query Plan Augmentation," Mathematics, 12, 2024].

[24] N. Karri, “AI-Powered Query Optimization,” International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 2, no. 1, pp. 63-71, %03/%30, 2021.

[25] D.Venkata Narasimha Raju, “AI-Powered Query Optimization in Multitenant Database Systems,” Journal of Computer Science and Technology Studies, vol. 7, no. 4, pp. 802-813, %05/%21, 2025.

[26] N. Sassi, and W. Jaziri, "Efficient AI-Driven Query Optimization in Large-Scale Databases: A Reinforcement Learning and Graph-Based Approach," Mathematics, 13, 2025].

[27] M. Sun, and L. Sun, “Optimization of artificial intelligence in localized big data real-time query processing task scheduling algorithm,” Frontiers in Physics, vol. Volume 12 - 2024, 2024-October-24, 2024.

Group Contribution

Student ID	Name	Contribution
22067128	Liang Yrenn Da	100%
22071112	Yeoh Jie Long	100%
22039069	Ooi Jin Yon	100%